

LOS SEIS OBJETIVOS, Y COMO LOS DEFIENDO

Seccion 2.2 del enunciado · Mini-JARVIS · Britany Macias

Cada objetivo con lo que hice, donde esta la prueba y la frase para decirlo

OBJETIVO 1

Explicar y evidenciar los conceptos de tokenizacion, embeddings, positional encoding y self-attention

LO QUE HICE

- Un modulo de exploracion que carga modelos reales y ensena los cuatro conceptos con la salida del propio programa, no con dibujos de libro.
- Tokenizacion con el tokenizador real de Qwen2.5, que es uno de los modelos que la aplicacion usa de verdad.
- Embeddings, positional encoding y self-attention con BETO, que se descarga completo a la laptop y si se puede abrir por dentro.
- Dos imagenes generadas por el programa: el mapa de atencion y el de posiciones.

DONDE ESTA LA PRUEBA

exploration/transformer_lab.py	el laboratorio entero, 852 lineas
linea 289	nivel 1: tokenizacion
linea 396	nivel 2: embeddings y self-attention
linea 559	nivel 3: positional encoding
linea 492	comprobacion de que cada fila de atencion suma 1.0
exploration/mapa_atencion.png	quien mira a quien
exploration/mapa_posiciones.png	como el modelo codifica el orden

LO QUE DIGO

El laboratorio trabaja en dos niveles. Para los tokens uso el tokenizador real de uno de mis modelos: son pocos megas y no hacen falta los pesos. Para mirar dentro uso BETO, porque el modelo con el que habla Elena es de otra empresa y esta cerrado: devuelve texto por una API y no hay forma de pedirle sus pesos de atencion. Los cuatro conceptos salen de ejecutar el programa, no de copiarlos.

Si te piden el positional encoding: pongo «perro gato» y «gato perro», comparo el vector de «perro» en las dos, y demuestro que no es el mismo. Si el orden no importara, seria identico.

OBJETIVO 2

Integrar un modelo de lenguaje preentrenado, por API en la nube o en local, dentro de una aplicacion funcional

LO QUE HICE

- Las dos cosas, no una. El enunciado permitia elegir; implemente ambas.
- En la nube: Qwen3.8 como modelo principal y Llama-3.3-70B como alternativo, intercambiables desde la propia ventana sin reiniciar.
- En local: cuando se cae la red, las tres piezas cambian solas a modelos que corren en la laptop, y la ventana avisa del cambio.
- No entrene ningun modelo. El enunciado dice explicitamente que no hacia falta.

DONDE ESTA LA PRUEBA

core/llm_engine.py	el motor de la nube
core/modo_local.py	el respaldo que corre en la maquina
config.py	los dos modelos declarados en un solo sitio
gui/desktop_app.py	el selector que los cambia en caliente

LO QUE DIGO

El principal es la nube porque da mejores respuestas. El local es un respaldo que entra solo cuando se cae la red. Elegi dos modelos de familias distintas por escarmiento: a tres dias de la entrega el proveedor retiro el alternativo que tenia puesto, probe los ciento sesenta y nueve del catalogo y solo veinte respondian. Si los dos vienen de la misma familia, un problema del proveedor se los lleva a la vez.

Detalle que suma: el cambio a local distingue entre quedarse sin red y tener la clave mal escrita. Si es la clave no cambia, porque eso taparia un error de configuracion en lugar de enseñarlo.

OBJETIVO 3

Implementar un pipeline de voz completo: reconocimiento de voz, procesamiento con el LLM y síntesis de voz

LO QUE HICE

- El recorrido entero funciona de extremo a extremo con voz real.
- Reconocimiento con Whisper large-v3, servido por Together AI.
- Procesamiento con el modelo de lenguaje, con memoria de los ultimos diez turnos y diez herramientas que puede pedir.
- Síntesis con edge-tts, voz es-AR-ElenaNeural.
- Un orquestador que coordina las tres piezas y mantiene el estado.

DONDE ESTA LA PRUEBA

core/audio_capture.py	microfono -> audio en memoria
core/stt_client.py	audio -> texto
core/llm_engine.py	texto -> respuesta del modelo
core/tts_engine.py	respuesta -> voz reproducida
core/orchestrator.py	la maquina de estados que lo coordina todo
core/memory.py	los diez turnos y su truncado

LO QUE DIGO

Micro, Whisper que lo pasa a texto, el modelo de lenguaje con la memoria y las herramientas, edge-tts que lo convierte en audio. Todo lo coordina el orquestador, y la ventana solo muestra el estado. La grabacion nunca toca el disco: vive en memoria, se transcribe y se descarta.

OBJETIVO 4

Disenar una interfaz que comunique el estado del asistente y sea agradable de usar

LO QUE HICE

- Cuatro estados mas el de error, cada uno con su color y su forma propia.
- El estado nunca se comunica solo por color: siempre hay texto que lo acompaña.
- Se puede hablar con el boton o con la barra espaciadora.
- Controles pensados para la demostracion: selector de modelo, temperatura, top_p y contador de memoria.
- El laboratorio del Transformer integrado en la misma ventana.

DONDE ESTA LA PRUEBA

gui/desktop_app.py	la ventana entera
core/orchestrator.py	linea 52: los estados, definidos en un solo sitio
tests/test_paleta_estados.py	prueba automatica de que cada estado se distingue

LO QUE DIGO

La ventana tiene dos columnas: a la izquierda un indicador que cambia segun el estado, a la derecha la conversacion y los controles. Los estados son reposo, escuchando, pensando y respondiendo, mas el de

atencion cuando algo falla. Cada uno cambia el color y tambien la forma, para que se distingan sin depender del color, y siempre van acompañados de texto.

OBJETIVO 5

Aplicar buenas practicas: manejo de errores, credenciales seguras, control de versiones y documentacion tecnica

LO QUE HICE

- Manejo de errores: cada fallo previsible tiene su tipo propio y su mensaje propio, y ninguno detiene la aplicacion.
- Credenciales: la clave vive en un archivo que git ignora. Lo compruebo sobre el historial completo, no solo sobre el ultimo commit.
- Control de versiones: 81 commits repartidos por tareas, no un unico envio final.
- Documentacion: README con la instalacion paso a paso, informe tecnico, y una carpeta de control interno con el plan y las decisiones.
- 189 pruebas automaticas que corren en doce segundos, sin red y sin gastar API.

DONDE ESTA LA PRUEBA

core/audio_capture.py	MicrofonoNoDisponible, GrabacionInvalida, GrabacionVacía
core/llm_engine.py	SinConexionLLM, ModeloDemasiadoLento, CredencialRechazadaLLM, RespuestaInvalida
config.py linea 41	la clave se lee del entorno, nunca del código
.gitignore linea 2	.env excluido del repositorio
tests/	siete archivos, 189 pruebas
README.md	instalacion, uso y estructura

LO QUE DIGO

Cada fallo previsible tiene su propio tipo de error y su propio mensaje, porque un mensaje que senala la causa equivocada es peor que no dar ninguno. La clave de la API nunca esta en el código: se lee de un archivo local que git ignora, y lo verifico recorriendo el historial entero. Y las pruebas corren sin red y sin microfono, asi que cualquiera puede ejecutarlas.

Si te preguntan cuantas pruebas: 189, y se ejecutan con «pytest» en unos doce segundos. Puedes correrlas en vivo.

OBJETIVO 6

Reflexionar sobre las limitaciones, los sesgos y las consideraciones eticas del uso de LLMs en aplicaciones reales

LO QUE HICE

- Documente las limitaciones en el informe en lugar de esconderlas.
- El asistente avisa al arrancar de que es una inteligencia artificial y de que puede equivocarse, y lo lleva tambien en su system prompt.
- La voz de quien habla no se guarda en ningun momento.
- Todo lo que toca el sistema pasa por listas blancas: el modelo propone, pero decidir que se ejecuta le corresponde al programa.
- Cite explicitamente los modelos y proveedores usados.

DONDE ESTA LA PRUEBA

docs/informe-tecnico.md	seccion de limitaciones y de etica
core/llm_engine.py	el system prompt con el aviso
gui/desktop_app.py	el aviso que sale al arrancar
tools/system_skills.py	la calculadora sin evaluar, las listas blancas

LO QUE DIGO

La limitacion mas seria es que el modelo puede inventarse un dato y decirlo con el mismo tono de seguridad que uno cierto. Eso no se arregla con codigo, porque no es un fallo del programa sino de como funciona un modelo de lenguaje. Lo que si hice fue avisarlo. Ademas el modelo principal es cerrado: no puedo ver sus pesos ni saber con que datos lo entrenaron, asi que tampoco puedo auditar sus sesgos, y eso hay que decirlo.

Decir tu las limitaciones suma. Que te las saquen ellos, resta.

SI TE PREGUNTAN POR ALGO QUE NO HICISTE

No inventes. Di: «eso no lo medi; lo que si compruebo es esto otro», y llevalo a algo que si hiciste. Reconocer un limite y redirigir a evidencia real demuestra que sabes distinguir lo que verificaste de lo que no. Inventar es lo unico que puede hundirte de verdad.